

You are an autonomous senior full-stack engineer, AI engineer, product architect, and UI/UX engineer.

Your task is to independently design and build a complete working prototype for a Razorpay buildathon challenge:

TRACK 03 — AI REVENUE RECOVERY

Goal:

Build an AI agent that detects revenue at risk, determines the appropriate intervention, and executes a bounded recovery workflow.

The system must handle revenue leakage scenarios such as:

- Payment failures
- Checkout abandonment
- Failed subscriptions
- Overdue B2B receivables
- Mandate/payment retry failures
- Promise-to-pay tracking

The core principle is:

Detect → Diagnose → Decide → Act → Verify → Stop → Audit

The solution must NOT merely identify problems. It must demonstrate actual/measurable money recovered across a batch of at-risk transactions/customers.

The final product should feel like a polished production-grade fintech AI operations platform, while using simulated/mock payment data and integrations for the buildathon.

=====

0. AUTONOMOUS EXECUTION RULE

=====

IMPORTANT:

Do NOT ask me questions during implementation.

Do NOT stop because requirements are ambiguous.

Do NOT ask me to choose:

- framework
- database
- UI library
- AI model
- architecture
- folder structure
- deployment method
- mock data format
- workflow design

Instead, make technically sensible decisions yourself.

Use the following defaults unless there is a compelling technical reason not to:

Frontend:

- React
- TypeScript
- Vite
- Tailwind CSS
- Recharts
- Lucide icons

Backend:

- Python
- FastAPI
- SQLAlchemy
- PostgreSQL

AI:

- LLM-based decision/diagnostic layer where useful
- deterministic rules as the safety layer
- structured JSON outputs from the AI
- fallback deterministic reasoning if an LLM/API key is unavailable

Background processing:

- FastAPI background tasks or a lightweight worker architecture
- no unnecessary infrastructure

Development:

- Docker Compose
- .env configuration
- seeded database
- mock integrations

If an external API requires credentials, implement a mock adapter and make the real integration replaceable through an interface.

The application **MUST** run locally without requiring paid external services.

=====

1. PRODUCT VISION

=====

Build a platform called:

RevenueRecover AI

The platform continuously analyzes payment/revenue events and identifies money that is likely to be lost.

For every revenue-risk case, the agent should answer:

1. What went wrong?
2. How much revenue is at risk?
3. Why did it happen?
4. What intervention should be attempted?
5. Why was this intervention selected?
6. What action should be executed?
7. What is the maximum number of attempts allowed?
8. When should the workflow stop?
9. Did the action recover money?
10. What happened afterward?

The platform should demonstrate that AI closes the loop:

Revenue Risk



Root Cause



Recovery Strategy



Bounded Action



Payment/Recovery Result



Revenue Recovered

↓

Audit Trail

=====

2. CORE BUSINESS REQUIREMENT

=====

The application must demonstrate measurable revenue recovery.

Create a simulated batch of revenue-risk cases.

Example:

100 at-risk transactions

₹25,00,000 total revenue at risk

↓

AI analyzes all 100

↓

Classifies causes

↓

Chooses interventions

↓

Executes simulated recovery actions

↓

Some payments succeed

↓

Some remain failed

↓

Some are escalated

↓

Dashboard reports:

Total revenue at risk

Total recovered

Recovery rate

Number of successful recoveries

Number of interventions

Cost/attempt metrics if applicable

Amount escalated

Amount permanently lost

The most important metric is:

MONEY RECOVERED

Do not make the product merely a chatbot.

The agent must perform actual workflow orchestration.

=====

3. SUPPORTED REVENUE-RISK SCENARIOS

=====

Implement at least these six scenarios:

A. PAYMENT DEGRADATION

A customer's previously successful payment behavior starts degrading.

Example:

Customer:

ACME Pvt Ltd

Historical:

95% payment success

Recent:

60% payment success

Failure pattern:

- bank decline
- insufficient funds
- network failures
- repeated retries

Agent must:

- detect degradation
- identify likely root cause
- determine appropriate retry strategy
- execute bounded retries
- stop after the configured retry limit
- escalate when appropriate
- record all actions

B. CHECKOUT DROP-OFF RECOVERY

A user reaches checkout but does not complete payment.

Data should include:

- checkout started
- amount
- timestamp
- payment method
- device
- geography
- previous customer behavior
- cart/order information

Agent should determine likely cause using available signals.

Possible interventions:

- payment retry link
- alternative payment method
- reminder
- personalized message
- limited-time recovery prompt

Do NOT repeatedly spam the customer.

Implement frequency limits.

C. FAILED SUBSCRIPTION RECOVERY

A recurring subscription payment fails.

Examples:

- card expired
- insufficient funds

- bank decline
- mandate failure
- payment method unavailable

Agent should:

- diagnose failure
- determine whether retry is appropriate
- schedule retry
- suggest/update payment method when appropriate
- stop after maximum attempts
- escalate if recovery fails

D. B2B RECEIVABLES CHASER

An invoice becomes overdue.

Data:

- customer
- invoice amount
- due date
- days overdue
- historical payment behavior
- previous promises
- customer segment
- account owner

Agent should classify:

LOW RISK

→ automated reminder

MEDIUM RISK

→ personalized follow-up

HIGH RISK

→ escalation to account manager

CRITICAL

→ management/manual review

The agent must not endlessly chase customers.

Implement stopping rules.

E. MANDATE RETRY SEQUENCER

Recurring payment/mandate fails.

Agent should select a retry sequence.

Example:

Attempt 1

↓

wait

↓

Attempt 2

↓

wait

↓

Attempt 3

↓

alternate payment method

↓

customer notification

↓

stop/escalate

The retry schedule should depend on failure type.

For example:

- temporary network failure → retry soon
- insufficient funds → retry later
- invalid payment method → request payment method update
- permanent decline → stop retries

F. PROMISE-TO-PAY TRACKER

For overdue invoices, a customer may promise to pay on a certain date.

Store:

- promise date
- promised amount
- actual payment
- reminder history
- missed promise count

Agent should:

Promise created

↓

Track due date

↓

Payment received?

YES → mark recovered

NO → trigger next intervention

↓

Repeated missed promise?

YES → escalate

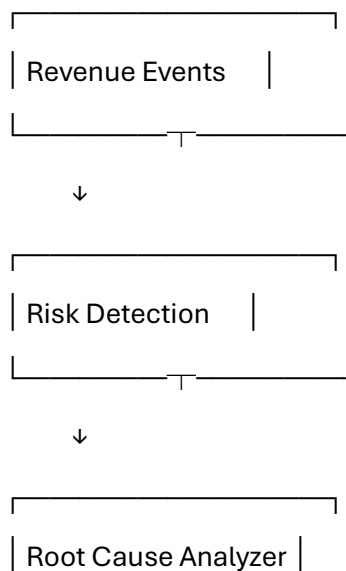
=====

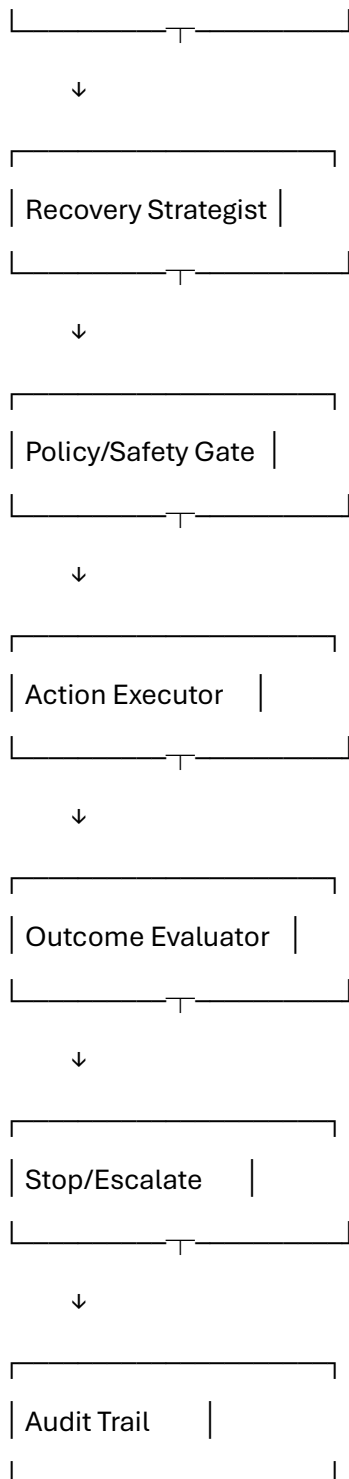
4. AGENT ARCHITECTURE

=====

Build the agent as a workflow/state-machine rather than a single unrestricted LLM.

Recommended architecture:





Use explicit states.

Example:

DETECTED

- DIAGNOSING
- STRATEGY_SELECTED
- POLICY_CHECK
- ACTION_PENDING
- ACTION_EXECUTED
- VERIFYING
- RECOVERED

or:

- DETECTED
- DIAGNOSING
- STRATEGY_SELECTED
- ACTION_EXECUTED
- FAILED
- RETRY_PENDING
- RETRY
- ESCALATED

or:

- DETECTED
- DIAGNOSING
- UNSAFE/NOT_ELIGIBLE
- STOPPED

=====

5. AI DECISION ENGINE

=====

Implement an AI decision engine.

The AI should receive structured case information such as:

```
{  
  customer,  
  transaction,  
  amount,  
  historical_success_rate,  
  recent_failures,  
  failure_reason,  
  payment_method,  
  customer_segment,  
  days_overdue,  
  previous_interventions,  
  previous_promises,  
  retry_count  
}
```

The AI should output structured data:

```
{  
  "risk_score": 0-100,  
  "risk_level": "LOW|MEDIUM|HIGH|CRITICAL",  
  "root_cause": "...",  
  "confidence": 0-1,  
  "recommended_action": "...",  
  "reasoning": "...",  
  "expected_recovery": 0,  
  "max_attempts": 3,  
  "should_escalate": false,  
  "stop_reason": null  
}
```

}

IMPORTANT:

The LLM must NOT have unrestricted control over financial actions.

The LLM proposes.

A deterministic policy engine approves/rejects.

Architecture:

LLM

↓

Structured decision

↓

Policy engine

↓

Approved action

↓

Executor

If the LLM is unavailable:

use deterministic rules to generate the same structure.

=====

6. POLICY / SAFETY ENGINE

=====

Implement hard-coded safety constraints.

The AI cannot override these.

Examples:

MAX_RETRIES = 3

MAX_CUSTOMER_CONTACTS_PER_7_DAYS = 2

MAX_TOTAL_RECOVERY_ATTEMPTS_PER_CASE = 3

DO_NOT_RETRY_PERMANENT_FAILURE = true

ESCALATE_AFTER_REPEATED_FAILURES = true

REQUIRE_MANUAL_REVIEW_ABOVE_AMOUNT = configurable

DO_NOT_CONTACT_CUSTOMER_OUTSIDE_ALLOWED_HOURS = true

STOP_IF_PAYMENT_ALREADY_SUCCESSFUL = true

STOP_IF_CUSTOMER_OPTOUT = true

STOP_IF_CASE_IS_CLOSED = true

STOP_IF_RECOVERY_AMOUNT_REACHED = true

Every action must pass through:

PolicyEngine.validate(action)

Possible outputs:

APPROVED

REJECTED

REQUIRES_MANUAL_REVIEW

STOP_WORKFLOW

=====

7. ROOT-CAUSE CLASSIFICATION

=====

Create a root-cause taxonomy.

Examples:

PAYMENT_FAILURE:

- insufficient_funds
- bank_decline
- expired_card
- invalid_payment_method
- network_error
- authentication_failure
- fraud/risk_block
- unknown

SUBSCRIPTION:

- expired_payment_method
- mandate_failure
- insufficient_funds
- recurring_payment_decline

CHECKOUT:

- user_abandoned
- payment_method_failure
- technical_error
- timeout
- unknown

RECEIVABLE:

- invoice_overdue
- missed_promise
- customer_delay
- dispute
- account_issue

Each cause should map to possible interventions.

=====

8. INTERVENTION STRATEGY

=====

Create a strategy engine.

Example mapping:

network_error

→ retry payment

insufficient_funds

→ delayed retry

expired_card

→ payment-method-update notification

mandate_failure

→ mandate retry + notification

checkout abandonment

→ recovery reminder

repeated invoice delay

→ personalized receivables escalation

missed promise-to-pay

→ follow-up + escalation

Each strategy must include:

- action
- reason
- expected recovery
- maximum attempts
- cooldown
- stopping condition
- escalation condition

=====

9. MOCK PAYMENT INFRASTRUCTURE

=====

Build mock adapters that behave like real integrations.

Create interfaces:

PaymentGateway

SubscriptionService

NotificationService

CRMService

InvoiceService

Example:

PaymentGateway.retry_payment(transaction_id)

PaymentGateway.get_payment_status(transaction_id)

NotificationService.send_sms(...)

NotificationService.send_email(...)

InvoiceService.get_invoice(...)

CRMService.create_escalation(...)

The application should use mock implementations.

Make them deterministic enough that demos are reproducible.

Some payments should succeed after recovery.

Some should fail permanently.

Some should require escalation.

This allows the dashboard to demonstrate actual recovered money.

=====

10. DATABASE MODEL

=====

Create PostgreSQL models.

At minimum:

Customer

Transaction

PaymentAttempt

Subscription

Invoice

RevenueRiskCase

RiskAssessment

RecoveryAction

RecoveryWorkflow

PromiseToPay

Notification

AuditEvent

AgentDecision

Suggested fields:

Customer:

- id

- name

- email

- phone

- segment

- lifetime_value

- risk_score

Transaction:

- id

- customer_id

- amount

- currency

- status

- payment_method

- failure_reason

- created_at

RevenueRiskCase:

- id

- customer_id

- source_type

- transaction_id

- invoice_id

- amount_at_risk

- recovered_amount

- status

- risk_level

- root_cause

- created_at

- updated_at

RecoveryAction:

- id

- case_id

- action_type

- status
- reason
- attempted_at
- result
- recovered_amount

AuditEvent:

- id
- case_id
- actor
- event_type
- timestamp
- details
- policy_result

PromiseToPay:

- id
- invoice_id
- promised_amount
- promise_date
- status
- fulfilled_at

=====

11. API DESIGN

=====

Implement REST APIs.

Dashboard:

GET /api/dashboard/summary

GET /api/dashboard/recovery

GET /api/dashboard/risk-distribution

Cases:

GET /api/cases

GET /api/cases/{case_id}

POST /api/cases/{case_id}/run

POST /api/cases/{case_id}/approve

POST /api/cases/{case_id}/stop

POST /api/cases/{case_id}/retry

Agent:

POST /api/agent/analyze/{case_id}

POST /api/agent/run/{case_id}

GET /api/agent/decisions/{case_id}

Actions:

GET /api/actions

GET /api/actions/{action_id}

Audit:

GET /api/audit/{case_id}

Batch:

POST /api/batch/analyze

POST /api/batch/run

GET /api/batch/{batch_id}

Simulation:

POST /api/simulation/seed

POST /api/simulation/run

=====

12. BATCH RECOVERY

=====

This is a critical part of the challenge.

Implement a batch-processing workflow.

Example:

POST /api/batch/run

The system should:

1. Fetch all unresolved revenue-risk cases.
2. Calculate risk.
3. Diagnose each case.
4. Select intervention.
5. Run policy validation.
6. Execute approved actions.
7. Simulate payment/recovery outcome.
8. Update recovered revenue.
9. Schedule/perform next action where appropriate.
10. Stop workflows that reach stopping conditions.
11. Escalate cases requiring human intervention.
12. Write audit events.

Return:

```
{
  "batch_id": "...",
  "cases_processed": 100,
  "total_at_risk": 2500000,
  "recovered_amount": 1375000,
  "recovery_rate": 55,
  "successful_recoveries": 61,
  "failed_cases": 21,
  "escalated_cases": 18
}
```

=====

13. REVENUE CALCULATIONS

=====

Track money precisely.

Definitions:

total_revenue_at_risk

=

sum(amount_at_risk for eligible unresolved cases)

recovered_revenue

=

sum(actual recovered amount)

recovery_rate

=

recovered_revenue / total_revenue_at_risk * 100

Do not count a payment as recovered unless the simulated payment outcome becomes successful.

Do not double-count revenue.

If a ₹10,000 case is recovered:

at risk = ₹10,000

recovered = ₹10,000

If ₹4,000 is recovered:

at risk = ₹10,000

recovered = ₹4,000

If the case is permanently lost:

recovered = ₹0

=====

14. AUDIT TRAIL

=====

Every meaningful agent action must be auditable.

Record:

timestamp

case ID

agent

decision

risk score

root cause

recommended action

policy result

actual action

action result

recovery amount

next step

stop reason

Example:

14:31:05

CASE-1042

Risk detected:

₹18,000

Root cause:

Insufficient funds

Risk:

HIGH

AI recommendation:

Retry after cooldown

Policy:

APPROVED

Action:

Payment retry #1

Result:

FAILED

Next action:

Retry after 24 hours

14:31:08

Payment retry #2

Result:

SUCCESS

Recovered:

₹18,000

Workflow:

STOPPED

Stop reason:

Revenue fully recovered

This audit trail should be visible in the UI.

=====

15. DASHBOARD

=====

Create a polished fintech dashboard.

The main dashboard must immediately show:

TOTAL REVENUE AT RISK

TOTAL REVENUE RECOVERED

RECOVERY RATE

ACTIVE RECOVERY CASES

CASES ESCALATED

SUCCESSFUL RECOVERIES

FAILED RECOVERIES

Display monetary values in INR.

Example:

₹25.4L

Revenue at risk

₹13.7L

Recovered

54.1%

Recovery rate

61

Successful recoveries

Charts:

1. Revenue at Risk vs Recovered

2. Recovery by Intervention

Example:

Payment retry ₹4.2L

Checkout recovery ₹2.1L

Subscription ₹3.4L

Receivables ₹3.1L

Mandate retry ₹0.9L

3. Revenue risk by category

4. Recovery funnel

Detected

→ Diagnosed

→ Intervention

→ Retry

→ Recovered

5. Failure reasons

6. Recovery trend over time

=====

16. CASE MANAGEMENT UI

=====

Create a cases page.

Columns:

Case ID

Customer
Type
Amount at Risk
Risk
Root Cause
Recommended Action
Attempts
Status
Recovered
Last Action

Example:

CASE-1024
Acme Corp
Subscription
₹24,000
HIGH
Mandate Failure
Retry Mandate
2/3
ACTIVE
₹0

Clicking a case opens a detailed case view.

=====

17. CASE DETAIL PAGE

=====

Show:

Customer information

Revenue at risk

Recovered amount

Risk score

Root cause

AI recommendation

Confidence

Timeline

Previous payment attempts

Recovery actions

Policy decisions

Audit trail

Next scheduled action

Stopping conditions

Escalation status

Also provide an "Agent Reasoning" panel.

Example:

WHY THIS CASE IS AT RISK

Payment success rate fell from 94% to 51%
over the last 30 days.

WHY THIS ACTION

The failure reason is temporary and previous
successful payments suggest another attempt
has a reasonable probability of succeeding.

POLICY CHECK

Retry allowed:

YES

Attempts:

1 / 3

Contact frequency:

Within limit

Expected recovery:

₹18,000

=====

18. LIVE AGENT RUN UI

=====

Create an interface where the user can run the agent.

Button:

RUN RECOVERY AGENT

When clicked, show an animated workflow:

Detecting revenue risk...

↓

Analyzing payment history...

↓

Diagnosing root cause...

↓

Selecting intervention...

↓

Running policy checks...

↓

Executing recovery...

↓

Verifying payment...

↓

Calculating recovered revenue...

↓

Updating audit trail...

Then show the result.

Example:

RECOVERY SUCCESSFUL

₹18,000 recovered

Action:

Payment retry

Attempts:

2

Root cause:

Temporary bank failure

The workflow should feel like an AI agent actually executing work.

=====

19. HUMAN-IN-THE-LOOP

=====

Not every case should be autonomously executed.

Create cases requiring manual review.

Examples:

Amount > configurable threshold

Repeated failures

High-value customer

Dispute detected

Fraud/risk-related failure

Policy conflict

Low AI confidence

Show:

MANUAL REVIEW REQUIRED

Then provide:

Approve

Reject

Escalate

Stop

Record the human decision in the audit trail.

=====

20. STOPPING RULES

=====

This is a key judging criterion.

Implement explicit stopping rules.

Examples:

STOP when payment succeeds.

STOP when full amount is recovered.

STOP when maximum retries are reached.

STOP when customer opts out.

STOP when failure is classified as permanent.

STOP when policy prohibits further contact.

STOP when invoice is closed.

ESCALATE when:

- repeated failures
- high-value receivable
- low-confidence diagnosis
- dispute
- policy violation
- missed promise-to-pay

The UI should clearly show WHY a workflow stopped.

=====

21. NOTIFICATION SYSTEM

=====

Create mock:

SMS

Email

In-app notification

Do not actually send messages.

Store every notification.

Example:

SMS

"Your payment of ₹12,000 could not be completed.

Please update your payment method using the secure
payment link."

The UI should show:

Notification sent

Channel: SMS

Timestamp

Template

Status

Implement contact-frequency limits.

=====

22. AI EXPLAINABILITY

=====

Do not simply show:

"AI says retry."

Show useful structured reasoning.

Example:

Risk score: 82/100

Signals:

- 4 failures in last 7 days
- historical success rate: 93%
- recent success rate: 42%
- failure reason: insufficient funds
- customer has successfully paid similar amounts previously

Conclusion:

High probability of temporary payment degradation.

Recommended action:

Delayed retry.

Expected recovery:

₹15,000

Confidence:

89%

This should be generated by the AI/rule engine but rendered as structured UI.

=====

23. DEMO DATA

=====

Seed the database with realistic data.

Create at least:

100 customers

150+ transactions

30+ payment failures

20+ checkout abandonments

20+ failed subscriptions

20+ overdue invoices

10+ mandate failures

10+ promise-to-pay records

At least 50 revenue-risk cases.

Use realistic Indian fintech data.

Amounts should range from:

₹500

to

₹5,00,000+

Use INR.

Create a mixture of:

successful recovery

partial recovery

failed recovery

escalated recovery

manual review

stopped workflows

The seed should produce visually interesting dashboard numbers.

=====

24. DETERMINISTIC DEMO MODE

=====

Implement a DEMO MODE.

This is important.

The entire demo must work without:

- OpenAI API
- Razorpay credentials
- external payment gateways
- SMS providers
- email providers

Use deterministic mock services.

Provide:

POST /api/demo/reset

POST /api/demo/run

The demo should produce predictable results.

For example:

100 cases

₹24.8L at risk

₹14.2L recovered

57.3% recovery rate

The exact values may differ, but they must be internally consistent.

=====

25. RAZORPAY-LIKE INTEGRATION DESIGN

=====

Design the system so actual Razorpay APIs could be integrated later.

Create adapter interfaces such as:

PaymentProvider

SubscriptionProvider

InvoiceProvider

NotificationProvider

Then:

MockPaymentProvider

MockSubscriptionProvider

MockInvoiceProvider

MockNotificationProvider

The business logic must depend on interfaces rather than hard-coded mock implementation.

This demonstrates production-minded architecture.

Do NOT claim that the application is processing real Razorpay payments.

Clearly label simulated transactions as simulation/demo data.

=====

26. FRONTEND DESIGN

=====

Make the UI polished.

Visual direction:

- modern fintech SaaS
- dark/light neutral palette
- clean cards
- strong typography
- subtle animations
- responsive
- professional
- no unnecessary gradients

- no excessive rounded components
- high information density
- excellent empty/loading/error states

Navigation:

Dashboard

Revenue Risks

Recovery Cases

Agent Runs

Customers

Receivables

Subscriptions

Audit Trail

Settings

Dashboard should be the primary experience.

=====

27. BACKEND QUALITY

=====

Use:

- typed Pydantic schemas
- service layer
- repository/database layer where useful
- proper error handling
- logging
- environment configuration
- database migrations

- deterministic seed scripts
- modular agent architecture

Do not place all logic inside API route handlers.

Suggested backend structure:

backend/

app/

main.py

config.py

api/

dashboard.py

cases.py

agent.py

batch.py

audit.py

simulation.py

models/

customer.py

transaction.py

case.py

recovery.py

audit.py

schemas/

services/

risk_engine.py

root_cause_engine.py

strategy_engine.py

policy_engine.py

recovery_engine.py

notification_service.py

batch_service.py

agent/

orchestrator.py

decision_engine.py

prompts.py

state_machine.py

integrations/

payment_provider.py

mock_payment_provider.py

notification_provider.py

db/

tests/

=====

28. TESTING

=====

Write automated tests.

At minimum test:

1. Risk detection

2. Root-cause classification

3. Strategy selection

4. Retry limits

5. Permanent failure stopping

6. Customer contact limits

7. Successful payment recovery

8. Partial recovery

9. Escalation

10. Promise-to-pay failure

11. Revenue calculations

12. No double-counting

13. Audit trail creation

14. Batch execution

15. Policy engine

Example:

Given:

amount_at_risk = ₹10,000

retry_count = 0

failure = temporary

Then:

action = RETRY

Given:

retry_count = 3

Then:

action = STOP/ESCALATE

Given:

payment_status = SUCCESS

Then:

workflow = STOPPED

recovered_amount = transaction_amount

=====

29. OBSERVABILITY

=====

Every agent run should have:

run_id

case_id

start_time

end_time

duration

steps_executed

decision

actions

result

revenue_recovered

failure_reason

Create an Agent Runs page.

Example:

RUN-8921

100 cases

₹24.8L at risk

₹14.2L recovered

57.3% recovery

86 successful actions

14 escalations

Duration: 4.2 sec

=====

30. SECURITY / FINTECH SAFETY

=====

Do not expose:

- raw card numbers
- CVV
- secrets
- API credentials

Use synthetic customer/payment data.

Mask sensitive fields.

Never allow arbitrary AI-generated code or arbitrary tool execution.

Only expose explicitly implemented tools:

analyze_case
get_payment_status
retry_payment
send_notification
create_escalation
update_promise
close_case

The AI can select from these tools, but cannot invent new tools.

=====

31. TOOL-CALL MODEL

=====

Implement an agent tool registry.

Example:

```
TOOLS = {  
  
    "get_payment_history": ...,  
  
    "get_customer_profile": ...,  
  
    "get_invoice": ...,  
  
    "get_subscription": ...,  
  
    "get_payment_status": ...,  
  
    "retry_payment": ...,  
  
    "send_notification": ...,  
  
    "create_escalation": ...,  
  
    "create_promise_to_pay": ...,  
  
    "check_policy": ...  
  
}
```

The orchestrator controls which tools are available at each workflow stage.

=====

32. AGENT STATE MACHINE

=====

Implement something similar to:

```
class RecoveryState:
```

```
    DETECTED
```

```
    DIAGNOSING
```

```
    STRATEGY_SELECTED
```

```
    POLICY_CHECK
```

```
    ACTION_PENDING
```

```
    ACTION_EXECUTED
```

```
    VERIFYING
```

```
    RECOVERED
```

```
    RETRY_PENDING
```

```
    ESCALATED
```

```
    STOPPED
```

```
    FAILED
```

Transitions must be explicit.

Do not allow invalid transitions.

=====

33. IMPORTANT DEMO SCENARIO

=====

Create one particularly impressive demo case.

Example:

Customer:

Acme Technologies

Invoice:

₹1,20,000

Original due date:

10 days ago

Historical payment behavior:

Usually pays within 2 days

Recent behavior:

No payment

Previous promise:

₹1,20,000 by yesterday

Promise:

MISSED

Agent diagnosis:

High recovery probability but requires escalation.

Agent action:

1. Verify invoice
2. Check payment history
3. Detect missed promise

4. Calculate risk
5. Select personalized reminder
6. Check policy
7. Send reminder
8. Simulate partial payment ₹50,000
9. Detect remaining ₹70,000
10. Update case
11. Create follow-up
12. Simulate final payment
13. Mark recovered

Final result:

₹1,20,000 recovered

The timeline should make this visually impressive.

=====

34. SECOND IMPRESSIVE DEMO

=====

Create a failed subscription case.

Example:

Subscription:

₹4,999/month

Payment fails:

Insufficient funds

Agent:

Detects failure

↓

Analyzes customer history

↓

Determines delayed retry is appropriate

↓

Schedules retry

↓

Retry fails

↓

Second retry

↓

Payment succeeds

↓

₹4,999 recovered

↓

Stops workflow

Show every step.

=====

35. BATCH DEMO

=====

Create a "Run Full Recovery Batch" button.

When clicked:

Process all eligible cases.

Display live progress:

Processing 1/100

Processing 2/100

...

Processing 100/100

Then show:

RECOVERY COMPLETE

Revenue at risk:

₹24,80,000

Revenue recovered:

₹14,20,000

Recovery rate:

57.3%

Successful recoveries:

64

Escalated:

17

Stopped:

19

Average attempts:

1.8

The numbers must be calculated from actual seeded records, not hard-coded UI values.

=====

36. API DOCUMENTATION

=====

Make FastAPI Swagger documentation available.

Ensure all important endpoints have:

- descriptions
- request schemas
- response schemas
- examples

=====

37. README

=====

Create a high-quality README.

Include:

Project overview

Architecture

Agent workflow

System diagram

Tech stack

Database schema

API endpoints

AI decision process

Policy engine

Stopping rules

Recovery calculation

Demo instructions

Testing

Environment variables

Future Razorpay integration

Limitations of simulation

Example screenshots/placeholders

=====

38. DOCKER

=====

Create:

docker-compose.yml

Services:

frontend

backend

postgres

The application should start with:

docker compose up --build

Provide health checks.

=====

39. ENVIRONMENT VARIABLES

=====

Create:

.env.example

Include:

DATABASE_URL=

LLM_API_KEY=

LLM_MODEL=

DEMO_MODE=true

MAX_RETRIES=3

MAX_CONTACTS_PER_WEEK=2

MANUAL_REVIEW_THRESHOLD=

Do not require LLM_API_KEY for demo mode.

=====

40. ERROR HANDLING

=====

Implement graceful handling for:

database failure

LLM failure

payment provider failure

invalid case

duplicate action

already recovered case

policy rejection

timeout

invalid agent response

If the LLM produces invalid JSON:

1. attempt structured parsing

2. retry once if configured
3. fall back to deterministic decision engine

The application must remain usable.

=====

41. NO FAKE METRICS

=====

VERY IMPORTANT:

Do not hard-code dashboard metrics.

All metrics must be derived from database records.

For example:

`recovered_amount = SUM(recovery_actions.recovered_amount)`

`recovery_rate =`

`recovered_amount / total_at_risk * 100`

Cases:

`COUNT(revenue_risk_cases)`

Escalations:

`COUNT(cases WHERE status = ESCALATED)`

This is necessary because the judge should be able to interact with the system and see metrics change.

=====

42. BUILD ORDER

=====

Execute the implementation in this exact order.

PHASE 1:

Create repository structure.

PHASE 2:

Initialize backend.

PHASE 3:

Initialize frontend.

PHASE 4:

Configure PostgreSQL.

PHASE 5:

Create database models.

PHASE 6:

Create migrations.

PHASE 7:

Create seed/demo data.

PHASE 8:

Implement risk engine.

PHASE 9:

Implement root-cause engine.

PHASE 10:

Implement strategy engine.

PHASE 11:

Implement policy engine.

PHASE 12:

Implement mock payment/invoice/subscription services.

PHASE 13:

Implement agent orchestrator/state machine.

PHASE 14:

Implement AI decision layer.

PHASE 15:

Implement recovery execution.

PHASE 16:

Implement batch recovery.

PHASE 17:

Implement audit logging.

PHASE 18:

Implement backend APIs.

PHASE 19:

Implement dashboard.

PHASE 20:

Implement cases UI.

PHASE 21:

Implement case detail/timeline.

PHASE 22:

Implement agent execution UI.

PHASE 23:

Implement batch recovery UI.

PHASE 24:

Implement audit trail UI.

PHASE 25:

Implement charts.

PHASE 26:

Implement tests.

PHASE 27:

Implement Docker.

PHASE 28:

Run tests.

PHASE 29:

Run the application.

PHASE 30:

Fix all runtime/build/type errors.

PHASE 31:

Verify the complete demo flow from beginning to end.

=====

43. SELF-VERIFICATION

=====

After implementation, DO NOT simply state that the project is complete.

Actually verify:

- frontend builds
- backend starts
- database starts
- migrations work
- seed works
- dashboard loads
- API endpoints work
- agent can process a case
- policy engine works
- batch recovery works
- recovered amount updates
- audit events appear
- charts update
- tests pass
- Docker works

If something fails:

Diagnose it.

Fix it.

Run the relevant test again.

Continue until working.

Do not ask me for help unless the environment makes implementation literally impossible.

=====

44. FINAL DEMO FLOW

=====

The final application should support this exact judge flow:

STEP 1

Open dashboard.

Judge sees:

Revenue at Risk

Revenue Recovered

Recovery Rate

Active Cases

Escalations

STEP 2

Open Revenue Risks.

Judge sees various cases.

STEP 3

Open a failed payment case.

Judge sees:

Why it failed

Risk score

AI diagnosis

Recommended action

STEP 4

Click:

RUN AGENT

Judge sees:

Detect

→ Diagnose

→ Decide

→ Policy

→ Execute

→ Verify

→ Recover

STEP 5

Money recovered increases.

STEP 6

Open audit trail.

Judge sees every decision/action.

STEP 7

Run full batch.

Judge sees:

100 cases processed

₹X at risk

₹Y recovered

Z% recovery

STEP 8

Open an escalated case.

Judge sees why autonomous recovery stopped.

STEP 9

Open Agent Runs.

Judge sees complete execution metrics.

=====

45. JUDGING OPTIMIZATION

=====

Optimize specifically for these judging dimensions:

1. REAL BUSINESS VALUE

The product should clearly answer:

"How much money did we recover?"

2. AGENTIC BEHAVIOR

The system should actually:

observe

reason

decide

act

verify

adapt

stop

3. BOUNDED AUTONOMY

Demonstrate that AI is powerful but constrained by deterministic policies.

4. MEASURABLE OUTCOME

Show actual recovered revenue.

5. EXPLAINABILITY

Every decision should have a reason.

6. SAFETY

Retries, notifications and escalation must have limits.

7. AUDITABILITY

Every action must be recorded.

8. PRODUCT QUALITY

The UI should look like something a fintech operations team could actually use.

9. TECHNICAL QUALITY

Use modular architecture and replaceable integrations.

10. DEMOABILITY

The entire system should be demonstrable in 3–5 minutes.

=====

46. OPTIONAL ADVANCED FEATURES

=====

If the core implementation is complete and stable, add these features.

Do NOT compromise the core system for these.

A. Recovery probability

Predict:

$P(\text{recovery} \mid \text{action})$

Example:

Retry:

72%

SMS:

44%

Escalation:

61%

Choose the action with highest expected recovery subject to policy.

Expected value:

$\text{expected_recovery} =$

$\text{probability_of_success} * \text{amount_at_risk}$

B. Intervention optimization

Compare:

retry

email

SMS

payment-method-update

account-manager-escalation

Choose the intervention with highest expected recovery.

C. Customer segmentation

VIP

High value

Regular

Low value

D. Recovery cost

Track estimated intervention cost.

Calculate:

Net recovered revenue =

recovered revenue - recovery cost

E. Counterfactual recommendation

Show:

"Retrying now has expected recovery of ₹X."

"Waiting 24 hours increases expected recovery to ₹Y."

F. Recovery funnel.

=====

47. CRITICAL PRODUCT PRINCIPLE

=====

The product must NOT be:

"ChatGPT for failed payments."

It must be:

"An autonomous revenue recovery operating system."

The difference is:

A chatbot:

tells the operator what to do.

This system:

detects the issue,

diagnoses it,

chooses the intervention,

checks policy,

executes the action,

verifies the result,

recovers revenue,

knows when to stop,

escalates when necessary,

and records everything.

=====

48. FINAL DELIVERABLE

=====

At the end, the repository must contain:

- complete frontend
- complete backend
- database
- migrations
- seed data
- AI decision engine
- deterministic fallback engine
- agent orchestrator
- state machine
- policy engine
- mock integrations
- batch recovery engine
- audit system
- dashboard
- case management
- agent execution UI
- charts
- tests
- Docker Compose
- .env.example
- README

The application must be runnable locally.

Final command should ideally be:

`docker compose up --build`

Then:

`http://localhost:3000`

The demo should work immediately with seeded data.

=====

49. FINAL RESPONSE AFTER BUILD

=====

Once implementation is complete, provide a concise final report containing:

1. What was built
2. Architecture
3. AI/agent workflow
4. Revenue recovery logic
5. Safety/stopping rules
6. Batch recovery demonstration
7. Test results
8. How to run
9. Demo credentials if any
10. Known limitations

Do not ask follow-up questions.

Do not leave TODOs for core functionality.

Do not provide pseudo-code where working implementation is expected.

Build the complete project now.